

Manufacturing Operational Insights Report

On

Dockerized Marketing PHP Larvel Service

Course Name: DevOps Fundamentals

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	YASH YADAV	EN22CS3011113
2	AYUSH MISHRA	EN22EL301020
3	VISHAL	EN22CS3011091
4	ASHUTOSH KUMAR	EN22EL301019
5	HARSH MANDLOI	EN22ME304035
6	AYUSH MODI	EN21CS301196

Group Name: 13D10

Project Number: DO-26

Industry Mentor Name: Mr. VAIBHAV

University Mentor Name: PROP. AVNESH JOSHI

Academic Year:2025-2026

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used *(If applicable)*

Results & Output

Add the below details here:

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

Problem Statement & Objectives

1. Problem Statement: Containerize a Marketing service built with PHP Laravel using industry best practices for Docker. The project includes writing multi-stage Dockerfiles to optimize image size and security, alongside a Docker Compose configuration to orchestrate the application and its database dependencies. This ensures a consistent, portable development and production environment across the entire team.

2. Project Objectives:

- Containerize a PHP Laravel Marketing application using Docker
- Write a multi-stage Dockerfile to optimize image size and security
- Use Docker Compose to orchestrate multiple services (Nginx, PHP, MySQL, Redis)
- Deploy the application live on AWS EC2 Free Tier
- Ensure a consistent, portable environment for development and production
- Push the complete project to GitHub for version control

3. Scope of the Project:

The scope of the project, as outlined in the high-level design documentation, includes the following:

- **Containerization of the Application:** Developing a containerized version of a Marketing service built using the PHP Laravel 11.x framework.

- **Multi-Stage Dockerfile Development:** Creating a multi-stage Dockerfile designed to optimize the final image size and enhance security for production.
- **Service Orchestration:** Utilizing Docker Compose to orchestrate and manage a multi-container environment consisting of Nginx, PHP-FPM, MySQL, and Redis.
- **Cloud Deployment:** Deploying the fully containerized application live on an AWS EC2 instance (t3.micro) within the Mumbai region.
- **Infrastructure Configuration:** Setting up the necessary cloud environment, including an Ubuntu 24.04 LTS operating system and AWS Security Groups to manage inbound traffic.
- **Data Persistence:** Implementing Docker volumes (specifically db_data) to ensure that MySQL database data is preserved even when containers are restarted.
- **Environment Standardization:** Establishing a consistent and portable environment to be used across both development and production stages.
- **Version Control Integration:** Managing the complete project codebase and configuration through GitHub for version control.

Proposed Solution

The solution utilizes a containerized architecture managed by Docker Compose. It employs a multi-stage build process to ensure the production image is minimal and secure. The architecture is deployed on an AWS EC2 instance (t3.micro) running Ubuntu 24.04 LTS, making the service accessible via a public Elastic IP.

1. Key features:

- **Multi-stage Docker Build:** Optimized images for reduced size and enhanced security.
- **Orchestration:** Managed integration of four distinct services (Nginx, PHP-FPM, MySQL, Redis) using Docker Compose.

- **Persistent Storage:** Uses Docker volumes (db_data) to ensure MySQL data survives container restarts.
- **Cloud Deployment:** Live hosting on AWS EC2 (Mumbai region).
- **Version Controlled:** Entire codebase and configuration stored on GitHub.

2. Overall Architecture / Workflow:

USER / BROWSER

HTTP Request → http://13.126.33.236:80

↕ Port 80 (Public Internet)

AWS SECURITY GROUP (marketing-laravel-sg)

Firewall: Ports 22 (SSH), 80 (HTTP), 8000, 3306, 6379

↕ Allowed through Firewall

AWS EC2 Instance | t3.micro | Ubuntu 24.04 LTS | Elastic IP: 13.126.33.236

Docker Network: marketing_network (bridge driver)

NGINX	PHP-FPM	MYSQL	REDIS
Web Server	Laravel App	Database	Cache/Session
nginx:alpine	php:8.2-alpine	mysql:8.0	redis:alpine
Port: 80:80	Port: 9000	Port: 3306	Port: 6379
marketing_nginx	marketing_app	marketing_db	marketing_redis

Persistent Volume: db_data → MySQL data survives container restarts

- **WORKFLOW**

Step	Component	Action	Port
1	Browser / Client	User sends HTTP request to Elastic IP	80
2	AWS Security Group	Firewall validates and allows port 80 inbound traffic	80
3	Nginx Container	Receives request, acts as reverse proxy	80
4	PHP-FPM Container	Nginx forwards PHP requests via FastCGI protocol	9000
5	Laravel Application	Processes business logic, handles routing	-
6	MySQL Container	Laravel queries the database when needed	3306
7	Redis Container	Session read/write and cache operations	6379
8	Laravel Application	Prepares and returns HTTP response	-
9	Nginx Container	Sends response back to the client	80
10	Browser / Client	Renders the Laravel welcome page	-

3. Tools & Technologies Used:

Layer	Technology	Version	Purpose
Application	PHP Laravel	11.x	Backend Web Framework
Language	PHP	8.2	Server-side Language
Web Server	Nginx	Alpine (Latest)	Reverse Proxy & HTTP Server
Database	MySQL	8.0	Relational Database
Cache / Session	Redis	Alpine (Latest)	Cache, Sessions & Queues
Containerization	Docker	24.x	Container Runtime
Orchestration	Docker Compose	1.x	Multi-container Management
Cloud	AWS EC2	t3.micro	Cloud Virtual Machine
OS	Ubuntu	24.04 LTS	Server Operating System
Version Control	Git + GitHub	Latest	Source Code Management

Results & Output

